

[Gemini Deep Research](https://web.archive.org/web/20260426112138/https://ai.google.dev/gemini-api/docs/deep-research) ([/web/20260426112138/https://ai.google.dev/gemini-api/docs/deep-research](https://web.archive.org/web/20260426112138/https://ai.google.dev/gemini-api/docs/deep-research)) is now available in preview with collaborative planning, visualization, MCP support, and more.

Gemini 3 Developer Guide

We have updated our [Terms of Service](https://web.archive.org/web/20260426112138/https://ai.google.dev/gemini-api/terms) (<https://web.archive.org/web/20260426112138/https://ai.google.dev/gemini-api/terms>).

Gemini 3 is our most intelligent model family to date, built on a foundation of state-of-the-art reasoning. It is designed to bring any idea to life by mastering agentic workflows, autonomous coding, and complex multimodal tasks. This guide covers key features of the Gemini 3 model family and how to get the most out of it.

Try Gemini 3.1 Pro Preview (https://web.archive.org/web/20260426112138/https://aistudio.google.com/prompts/new_chat?model=gemini-3.1-pro-preview)

Try Gemini 3 Flash (https://web.archive.org/web/20260426112138/https://aistudio.google.com/prompts/new_chat?model=gemini-3-flash-preview)

Try Nano Banana 2 (https://web.archive.org/web/20260426112138/https://aistudio.google.com/prompts/new_chat?model=gemini-3.1-flash-image-preview)

Explore our [collection of Gemini 3 apps](https://web.archive.org/web/20260426112138/https://aistudio.google.com/app/apps?source=showcase&showcaseTag=gemini-3)

(<https://web.archive.org/web/20260426112138/https://aistudio.google.com/app/apps?source=showcase&showcaseTag=gemini-3>) to see how the model handles advanced reasoning, autonomous coding, and complex multimodal tasks.

Get started with a few lines of code:

Meet the Gemini 3 series

Gemini 3.1 Pro is best for complex tasks that require broad world knowledge and advanced reasoning across modalities.

Gemini 3 Flash is our latest 3-series model, with Pro-level intelligence at the speed and pricing of Flash.

Nano Banana Pro (also known as Gemini 3 Pro Image) is our highest quality image generation model, and Nano Banana 2 (also known as Gemini 3.1 Flash Image) is the high-volume, high-efficiency, lower price-point equivalent.

Gemini 3.1 Flash-Lite is our workhorse model built for cost-efficiency model and high-volume tasks.

All Gemini 3 models are currently in preview.

Model ID	Context Window (In / Out)	Knowledge Cutoff	Pricing (Input / Output)*
gemini-3.1-flash-lite-preview	1M / 64k	Jan 2025	\$0.25 (text, image, video), \$0.50 (audio) / \$1.50
gemini-3.1-flash-image-preview	128k / 32k	Jan 2025	\$0.25 (Text Input) / \$0.067 (Image Output)**
gemini-3.1-pro-preview	1M / 64k	Jan 2025	\$2 / \$12 (<200k tokens) \$4 / \$18 (>200k tokens)
gemini-3-flash-preview	1M / 64k	Jan 2025	\$0.50 / \$3
gemini-3-pro-image-preview	65k / 32k	Jan 2025	\$2 (Text Input) / \$0.134 (Image Output)**

** Pricing is per 1 million tokens unless otherwise noted. ** Image pricing varies by resolution. See the [pricing page](https://ai.google.dev/gemini-api/docs/pricing) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/pricing) for details.*

For detailed limits, pricing, and additional information, see the [models page](https://ai.google.dev/gemini-api/docs/models/gemini) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/models/gemini).

New API features in Gemini 3

Gemini 3 introduces new parameters designed to give developers more control over latency, cost, and multimodal fidelity.

Thinking level

Gemini 3 series models use dynamic thinking by default to reason through prompts. You can use the `thinking_level` parameter, which controls the **maximum** depth of the model's internal reasoning process before it produces a response. Gemini 3 treats these levels as relative allowances for thinking rather than strict token guarantees.

If `thinking_level` is not specified, Gemini 3 will default to `high`. For faster, lower-latency responses when complex reasoning isn't required, you can constrain the model's thinking level to `low`.

Thinking Level	Gemini 3.1 Pro	Gemini 3.1 Flash-Lite	Gemini 3 Flash	Description
<code>minimal</code>	Not supported	Supported (Default)	Supported	Matches the "no thinking" setting for most queries. The model may think very minimally for complex coding tasks. Minimizes latency for chat or high throughput

Thinking Level	Gemini 3.1 Pro	Gemini 3.1 Flash-Lite	Gemini 3 Flash	Description
				applications. Note, <code>minimal</code> does not guarantee that thinking is off.
low	Supported	Supported	Supported	Minimizes latency and cost. Best for simple instruction following, chat, or high-throughput applications.
medium	Supported	Supported	Supported	Balanced thinking for most tasks.
high	Supported (Default, Dynamic)	Supported (Dynamic)	Supported (Default, Dynamic)	Maximizes reasoning depth. The model may take significantly longer to reach a first (non thinking) output token, but the output will be more carefully reasoned.

Important: You cannot use both `thinking_level` and the legacy `thinking_budget` parameter in the same request. Doing so will return a 400 error.

Media resolution

Gemini 3 introduces granular control over multimodal vision processing via the `media_resolution` parameter. Higher resolutions improve the model's ability to read fine text or identify small details, but increase token usage and latency. The `media_resolution` parameter determines the **maximum number of tokens allocated per input image or video frame**.

You can now set the resolution to `media_resolution_low`, `media_resolution_medium`, `media_resolution_high`, or `media_resolution_ultra_high` per individual media part or globally (via `generation_config`, global not available for ultra high). If unspecified, the model uses optimal defaults based on the media type.

Recommended settings

Media Type	Recommended Setting	Max Tokens	Usage Guidance
Images	<code>media_resolution_high</code>	1120	Recommended for most image analysis tasks to ensure maximum quality.
PDFs	<code>media_resolution_medium</code>	560	Optimal for document understanding; quality typically saturates at <code>medium</code> . Increasing to <code>high</code> rarely improves OCR results for standard documents.
Video (General)	<code>media_resolution_low</code> (or <code>media_resolution_medium</code>)	70 (per frame)	Note: For video, <code>low</code> and <code>medium</code> settings are treated identically (70 tokens) to optimize context usage. This is sufficient for most action recognition and description tasks.
Video (Text-heavy)	<code>media_resolution_high</code>	280 (per frame)	Required only when the use case involves reading dense text (OCR) or small details within video frames.

Note: The `media_resolution` parameter maps to different token counts depending on the input type. While images scale linearly (`media_resolution_low`: 280, `media_resolution_medium`: 560, `media_resolution_high`: 1120), Video is compressed more aggressively. For Video, both `media_resolution_low` and `media_resolution_medium` are capped at 70 tokens per frame, and `media_resolution_high` is capped at 280 tokens. See the full details on the [Media resolution](https://ai.google.dev/gemini-api/docs/media-resolution#token-counts) ([/web/20260426112138/https://ai.google.dev/gemini-api/docs/media-resolution#token-counts](https://web.archive.org/web/20260426112138/https://ai.google.dev/gemini-api/docs/media-resolution#token-counts)) page.

Temperature

For all Gemini 3 models, we strongly recommend keeping the temperature parameter at its default value of `1.0`.

While previous models often benefited from tuning temperature to control creativity versus determinism, Gemini 3's reasoning capabilities are optimized for the default setting. Changing the temperature (setting it below 1.0) may lead to unexpected behavior, such as looping or degraded performance, particularly in complex mathematical or reasoning tasks.

Thought signatures

Gemini 3 uses Thought signatures ([/web/20260426112138/https://ai.google.dev/gemini-api/docs/thought-signatures](https://web/20260426112138/https://ai.google.dev/gemini-api/docs/thought-signatures)) to maintain reasoning context across API calls. These signatures are encrypted representations of the model's internal thought process. To ensure the model maintains its reasoning capabilities you must return these signatures back to the model in your request exactly as they were received:

- **Function Calling (Strict):** The API enforces strict validation on the "Current Turn". Missing signatures will result in a 400 error.
- ★ **Note:** Circulation of thought signatures is required even when thinking_level (`#thinking_level`) is set to `minimal` for Gemini 3 Flash.
- **Text/Chat:** Validation is not strictly enforced, but omitting signatures will degrade the model's reasoning and answer quality.
- **Image generation/editing (Strict):** The API enforces strict validation on all Model parts including a `thoughtSignature`. Missing signatures will result in a 400 error.

Success: If you use the official SDKs ([Python](#), [Node](#), [Java](#)).

(<https://web.archive.org/web/20260426112138/https://ai.google.dev/gemini-api/docs/function-calling?example=meeting#thinking>) and standard chat history, Thought Signatures are handled automatically. You do not need to manually manage these fields.

Function calling (strict validation)

When Gemini generates a `functionCall`, it relies on the `thoughtSignature` to process the tool's output correctly in the next turn. The "Current Turn" includes all Model (`functionCall`) and User (`functionResponse`) steps that occurred since the last standard **User** text message.

- **Single Function Call:** The `functionCall` part contains a signature. You must return it.
- **Parallel Function Calls:** Only the first `functionCall` part in the list will contain the signature. You must return the parts in the exact order received.
- **Multi-Step (Sequential):** If the model calls a tool, receives a result, and calls *another* tool (within the same turn), **both** function calls have signatures. You must return **all** accumulated signatures in the history.

Text and streaming

For standard chat or text generation, the presence of a signature is not guaranteed.

- **Non-Streaming:** The final content part of the response may contain a `thoughtSignature`, though it is not always present. If one is returned, you should send it back to maintain best performance.
- **Streaming:** If a signature is generated, it may arrive in a final chunk that contains an empty text part. Ensure your stream parser checks for signatures even if the text field is empty.

Image generation and editing

For `gemini-3-pro-image-preview` and `gemini-3.1-flash-image-preview`, thought signatures are critical for conversational editing. When you ask the model to modify an image it relies on the `thoughtSignature` from the previous turn to understand the composition and logic of the original image.

- **Editing:** Signatures are guaranteed on the first part after the thoughts of the response (`text` or `inlineData`) and on every subsequent `inlineData` part. You must return all of these signatures to avoid errors.

Code examples

Multi-step Function Calling (Sequential)

The user asks a question requiring two separate steps (Check Flight -> Book Taxi) in one turn.

Step 1: Model calls Flight Tool.

The model returns a signature `<Sig_A>`

```
// Model Response (Turn 1, Step 1)
{
  "role": "model",
  "parts": [
    {
      "functionCall": { "name": "check_flight", "args": {...} },
      "thoughtSignature": "<Sig_A>" // SAVE THIS
    }
  ]
}
```

Step 2: User sends Flight Result

We must send back `<Sig_A>` to keep the model's train of thought.


```
// User Request (Turn 1, Step 2)
[
  { "role": "user", "parts": [{ "text": "Check flight AA100..." }] },
  {
    "role": "model",
    "parts": [
      {
        "functionCall": { "name": "check_flight", "args": {...} },
        "thoughtSignature": "<Sig_A>" // REQUIRED
      }
    ]
  },
  { "role": "user", "parts": [{ "functionResponse": { "name": "check_flight", "response": {...} } }] }
]
```

Step 3: Model calls Taxi Tool

The model remembers the flight delay via <Sig_A> and now decides to book a taxi. It generates a *new* signature <Sig_B>.

```
// Model Response (Turn 1, Step 3)
{
  "role": "model",
  "parts": [
    {
      "functionCall": { "name": "book_taxi", "args": {...} },
      "thoughtSignature": "<Sig_B>" // SAVE THIS
    }
  ]
}
```

```
]
}
```

Step 4: User sends Taxi Result

To complete the turn, you must send back the entire chain: <Sig_A> AND <Sig_B>.

```
// User Request (Turn 1, Step 4)
[
  // ... previous history ...
  {
    "role": "model",
    "parts": [
      { "functionCall": { "name": "check_flight", ... }, "thoughtSignature": "<Sig_A>" }
    ]
  },
  { "role": "user", "parts": [{ "functionResponse": {...} }] },
  {
    "role": "model",
    "parts": [
      { "functionCall": { "name": "book_taxi", ... }, "thoughtSignature": "<Sig_B>" }
    ]
  },
  { "role": "user", "parts": [{ "functionResponse": {...} }] }
]
```

Parallel Function Calling

The user asks: "Check the weather in Paris and London." The model returns two function calls in one response.

```
// User Request (Sending Parallel Results)
[
  {
    "role": "user",
    "parts": [
      { "text": "Check the weather in Paris and London." }
    ]
  },
  {
    "role": "model",
    "parts": [
      // 1. First Function Call has the signature
      {
        "functionCall": { "name": "check_weather", "args": { "city": "Paris" } },
        "thoughtSignature": "<Signature_A>"
      },
      // 2. Subsequent parallel calls DO NOT have signatures
      {
        "functionCall": { "name": "check_weather", "args": { "city": "London" } }
      }
    ]
  },
  {
    "role": "user",
    "parts": [
      // 3. Function Responses are grouped together in the next block
      {
        "functionResponse": { "name": "check_weather", "response": { "temp": "15C" } }
      },
      {
        "functionResponse": { "name": "check_weather", "response": { "temp": "12C" } }
      }
    ]
  }
]
```

```

    }
  ]
}
]
```

Text/In-Context Reasoning (No Validation)

The user asks a question that requires in-context reasoning without external tools. While not strictly validated, including the signature helps the model maintain the reasoning chain for follow-up questions.

```
// User Request (Follow-up question)
[
  {
    "role": "user",
    "parts": [{ "text": "What are the risks of this investment?" }]
  },
  {
    "role": "model",
    "parts": [
      {
        "text": "I need to calculate the risk step-by-step. First, I'll look at volatility...",
        "thoughtSignature": "<Signature_C>" // Recommended to include
      }
    ]
  },
  {
    "role": "user",
    "parts": [{ "text": "Summarize that in one sentence." }]
  }
]
```

```
}  
]
```

Image Generation & Editing

For image generation, signatures are strictly validated. They appear on the **first part** (text or image) and **all subsequent image parts**. All must be returned in the next turn.

```
// Model Response (Turn 1)  
{  
  "role": "model",  
  "parts": [  
    // 1. First part ALWAYS has a signature (even if text)  
    {  
      "text": "I will generate a cyberpunk city...",  
      "thoughtSignature": "<Signature_D>"  
    },  
    // 2. ALL InlineData (Image) parts ALWAYS have signatures  
    {  
      "inlineData": { ... },  
      "thoughtSignature": "<Signature_E>"  
    },  
  ]  
}  
  
// User Request (Turn 2 - Requesting an Edit)  
{  
  "contents": [  
    // History must include ALL signatures received
```

```

{
  "role": "user",
  "parts": [{ "text": "Generate a cyberpunk city" }]
},
{
  "role": "model",
  "parts": [
    { "text": "...", "thoughtSignature": "<Signature_D>" },
    { "inlineData": "...", "thoughtSignature": "<Signature_E>" },
  ]
},
// New User Prompt
{
  "role": "user",
  "parts": [{ "text": "Make it daytime." }]
}
]
}

```

Migrating from other models

If you are transferring a conversation trace from another model (e.g., Gemini 2.5) or injecting a custom function call that was not generated by Gemini 3, you will not have a valid signature.

To bypass strict validation in these specific scenarios, populate the field with this specific dummy string: `"thoughtSignature": "context_engineering_is_the_way_to_go"`

Structured Outputs with tools

Gemini 3 models allow you to combine [Structured Outputs](/web/20260426112138/https://ai.google.dev/gemini-api/docs/structured-output) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/structured-output) with built-in tools, including [Grounding with Google Search](/web/20260426112138/https://ai.google.dev/gemini-api/docs/google-search) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/google-search), [URL Context](/web/20260426112138/https://ai.google.dev/gemini-api/docs/url-context) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/url-context), [Code Execution](/web/20260426112138/https://ai.google.dev/gemini-api/docs/code-execution) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/code-execution), and [Function Calling](/web/20260426112138/https://ai.google.dev/gemini-api/docs/function-calling) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/function-calling).

Image generation

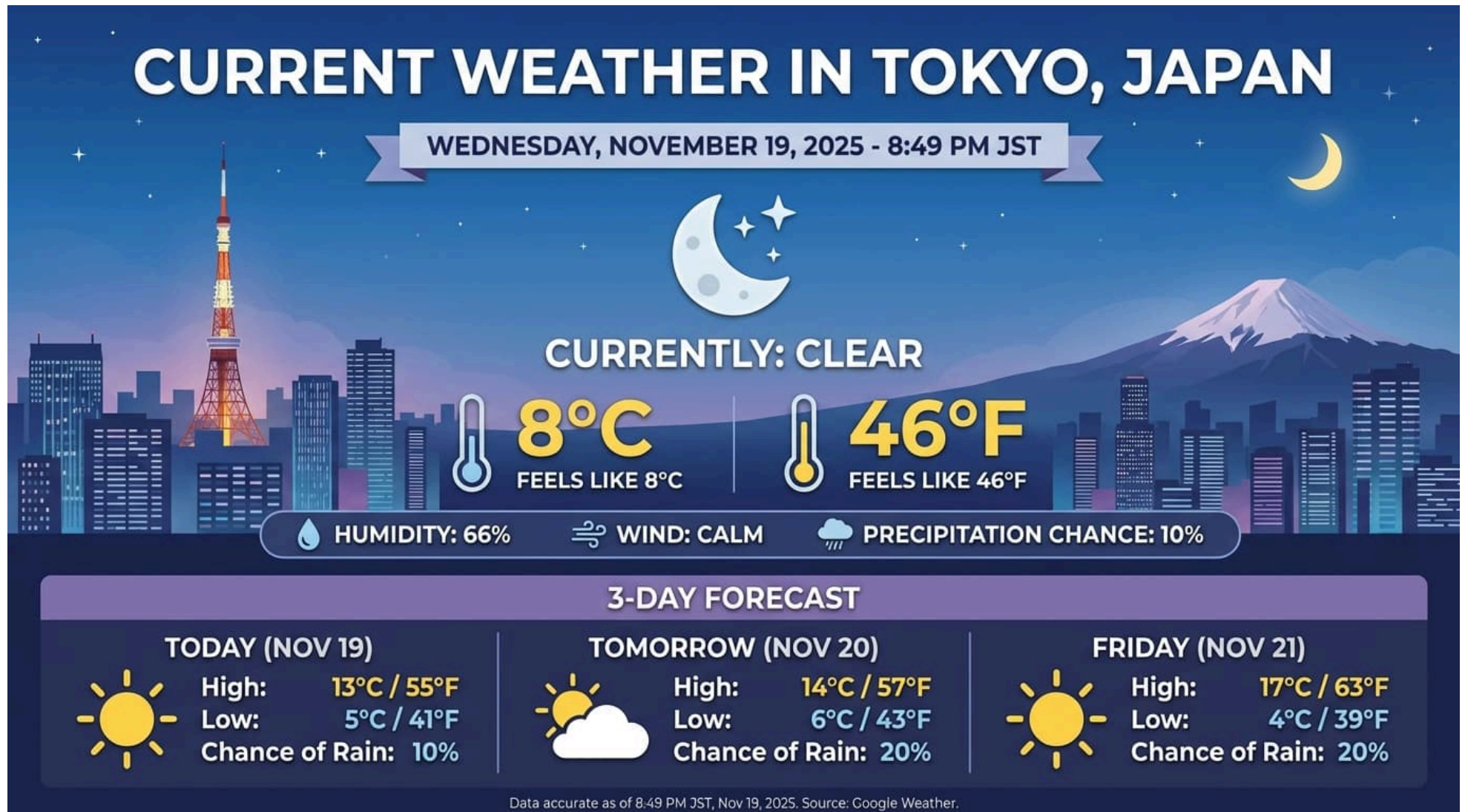
Gemini 3.1 Flash Image and Gemini 3 Pro Image let you generate and edit images from text prompts. It uses reasoning to "think" through a prompt and can retrieve real-time data—such as weather forecasts or stock charts—before using [Google Search](/web/20260426112138/https://ai.google.dev/gemini-api/docs/google-search) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/google-search) grounding before generating high-fidelity images.

New & improved capabilities:

- **4K & text rendering:** Generate sharp, legible text and diagrams with up to 2K and 4K resolutions.
- **Grounded generation:** Use the `google_search` tool to verify facts and generate imagery based on real-world information. Grounding with Google *Image* Search available for Gemini 3.1 Flash Image.
- **Conversational editing:** Multi-turn image editing by simply asking for changes (e.g., "Make the background a sunset"). This workflow relies on **Thought Signatures** to preserve visual context between turns.

For complete details on aspect ratios, editing workflows, and configuration options, see the [Image Generation guide](/web/20260426112138/https://ai.google.dev/gemini-api/docs/image-generation) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/image-generation).

Example Response



Code Execution with images

Gemini 3 Flash can treat vision as an active investigation, not just a static glance. By combining reasoning with [code execution](https://ai.google.dev/gemini-api/docs/code-execution) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/code-execution), the model formulates a plan, then writes and executes Python code to zoom in, crop, annotate, or otherwise manipulate images step-by-step to visually ground its answers.

Use cases:

- **Zoom and inspect:** The model implicitly detects when details are too small (e.g., reading a distant gauge or serial number) and writes code to crop and re-examine the area at higher resolution.
- **Visual math and plotting:** The model can run multi-step calculations using code (e.g., summing line items on a receipt, or generating a Matplotlib chart from extracted data).
- **Image annotation:** The model can draw arrows, bounding boxes, or other annotations directly onto images to answer spatial questions like "Where should this item go?".

To enable visual thinking, configure [Code Execution](https://ai.google.dev/gemini-api/docs/code-execution) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/code-execution) as a tool. The model will automatically use code to manipulate images when needed.

For more details on code execution with images, see [Code Execution](https://ai.google.dev/gemini-api/docs/code-execution#images) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/code-execution#images).

Multimodal function responses

[Multimodal function calling](https://ai.google.dev/gemini-api/docs/function-calling#multimodal) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/function-calling#multimodal) allows users to have function responses containing multimodal objects allowing for improved utilization of function calling capabilities of the model. Standard function calling only supports text-based function responses:

Combine built-in tools and function calling

Gemini 3 allows the use of built-in tools (like Google Search, URL context, and [more](/web/20260426112138/https://ai.google.dev/gemini-api/docs/tools) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/tools)) and custom [function calling](/web/20260426112138/https://ai.google.dev/gemini-api/docs/function-calling) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/function-calling) tools in the same API call, allowing for more complex workflows. Learn more on the [tool combinations](/web/20260426112138/https://ai.google.dev/gemini-api/docs/tool-combination) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/tool-combination) page.

Migrating from Gemini 2.5

Gemini 3 is our most capable model family to date and offers a stepwise improvement over Gemini 2.5. When migrating, consider the following:

- **Thinking:** If you were previously using complex prompt engineering (like chain of thought) to force Gemini 2.5 to reason, try Gemini 3 with `thinking_level: "high"` and simplified prompts.
- **Temperature settings:** If your existing code explicitly sets temperature (especially to low values for deterministic outputs), we recommend removing this parameter and using the Gemini 3 default of 1.0 to avoid potential looping issues or performance degradation on complex tasks.
- **PDF & document understanding:** If you relied on specific behavior for dense document parsing, test the new `media_resolution_high` setting to ensure continued accuracy.
- **Token consumption:** Migrating to Gemini 3 defaults may **increase** token usage for PDFs but **decrease** token usage for video. If requests now exceed the context window due to higher default resolutions, we recommend explicitly reducing the media resolution.
- **Image segmentation:** Image segmentation capabilities (returning pixel-level masks for objects) are not supported in Gemini 3 Pro or Gemini 3 Flash. For workloads requiring native image segmentation, we recommend continuing to utilize Gemini 2.5 Flash with

thinking turned off or [Gemini Robotics-ER 1.6](https://ai.google.dev/gemini-api/docs/robotics-overview) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/robotics-overview).

- **Computer Use:** Gemini 3 Pro and Gemini 3 Flash support [Computer Use](https://ai.google.dev/gemini-api/docs/computer-use) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/computer-use). Unlike the 2.5 series, you don't need to use a separate model to access the Computer Use tool.
- **Tool support:** [Combining built-in tools with function calling](https://ai.google.dev/gemini-api/docs/tool-combination) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/tool-combination) is now supported for Gemini 3 models. [Maps grounding](https://ai.google.dev/gemini-api/docs/maps-grounding) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/maps-grounding) is also now supported for Gemini 3 models.

OpenAI compatibility

For users utilizing the [OpenAI compatibility layer](https://ai.google.dev/gemini-api/docs/openai) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/openai), standard parameters (OpenAI's `reasoning_effort`) are automatically mapped to Gemini (`thinking_level`) equivalents.

Prompting best practices

Gemini 3 is a reasoning model, which changes how you should prompt.

- **Precise instructions:** Be concise in your input prompts. Gemini 3 responds best to direct, clear instructions. It may over-analyze verbose or overly complex prompt engineering techniques used for older models.
- **Output verbosity:** By default, Gemini 3 is less verbose and prefers providing direct, efficient answers. If your use case requires a more conversational or "chatty" persona, you must explicitly steer the model in the prompt (e.g., "Explain this as a friendly, talkative assistant").

- **Context management:** When working with large datasets (e.g., entire books, codebases, or long videos), place your specific instructions or questions at the end of the prompt, after the data context. Anchor the model's reasoning to the provided data by starting your question with a phrase like, "Based on the information above..."

Learn more about prompt design strategies in the [prompt engineering guide](https://ai.google.dev/gemini-api/docs/prompting-strategies) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/prompting-strategies).

FAQ

1. **What is the knowledge cutoff for Gemini 3?** Gemini 3 models have a knowledge cutoff of January 2025. For more recent information, use the [Search Grounding](https://ai.google.dev/gemini-api/docs/google-search) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/google-search) tool.
2. **What are the context window limits?** Gemini 3 models support a 1 million token input context window and up to 64k tokens of output.
3. **Is there a free tier for Gemini 3?** Gemini 3 Flash `gemini-3-flash-preview` and 3.1 Flash-Lite `gemini-3.1-flash-lite-preview` have free tiers in the Gemini API. You can try Gemini 3.1 Pro and 3 Flash for free in Google AI Studio, but there is no free tier available for `gemini-3.1-pro-preview` in the Gemini API.
4. **Will my old `thinking_budget` code still work?** Yes, `thinking_budget` is still supported for backward compatibility, but we recommend migrating to `thinking_level` for more predictable performance. Do not use both in the same request.
5. **Does Gemini 3 support the Batch API?** Yes, Gemini 3 supports the [Batch API](https://ai.google.dev/gemini-api/docs/batch-api) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/batch-api).
6. **Is Context Caching supported?** Yes, [Context Caching](https://ai.google.dev/gemini-api/docs/caching) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/caching) is supported for Gemini 3.
7. **Which tools are supported in Gemini 3?** Gemini 3 supports [Google Search](https://ai.google.dev/gemini-api/docs/google-search) (/web/20260426112138/https://ai.google.dev/gemini-api/docs/google-search), [Grounding with Google Maps](https://ai.google.dev/gemini-api/docs/grounding-with-google-maps)

([/web/20260426112138/https://ai.google.dev/gemini-api/docs/maps-grounding](https://web.archive.org/web/20260426112138/https://ai.google.dev/gemini-api/docs/maps-grounding)), [File Search](https://web.archive.org/web/20260426112138/https://ai.google.dev/gemini-api/docs/file-search) ([/web/20260426112138/https://ai.google.dev/gemini-api/docs/file-search](https://web.archive.org/web/20260426112138/https://ai.google.dev/gemini-api/docs/file-search)), [Code Execution](https://web.archive.org/web/20260426112138/https://ai.google.dev/gemini-api/docs/code-execution) ([/web/20260426112138/https://ai.google.dev/gemini-api/docs/code-execution](https://web.archive.org/web/20260426112138/https://ai.google.dev/gemini-api/docs/code-execution)), and [URL Context](https://web.archive.org/web/20260426112138/https://ai.google.dev/gemini-api/docs/url-context) ([/web/20260426112138/https://ai.google.dev/gemini-api/docs/url-context](https://web.archive.org/web/20260426112138/https://ai.google.dev/gemini-api/docs/url-context)). It also supports standard [Function Calling](https://web.archive.org/web/20260426112138/https://ai.google.dev/gemini-api/docs/function-calling) ([/web/20260426112138/https://ai.google.dev/gemini-api/docs/function-calling](https://web.archive.org/web/20260426112138/https://ai.google.dev/gemini-api/docs/function-calling)) for your own custom tools, and in [combination with built-in tools](https://web.archive.org/web/20260426112138/https://ai.google.dev/gemini-api/docs/tool-combination) ([/web/20260426112138/https://ai.google.dev/gemini-api/docs/tool-combination](https://web.archive.org/web/20260426112138/https://ai.google.dev/gemini-api/docs/tool-combination)).

8. What is gemini-3.1-pro-preview-customtools? If you are using **gemini-3.1-pro-preview** and the model ignores your custom tools in favor of bash commands, try the **gemini-3.1-pro-preview-customtools** model instead. More info [here](https://web.archive.org/web/20260426112138/https://ai.google.dev/gemini-api/docs/models/gemini-3.1-pro-preview#gemini-3.1-pro-preview-customtools) ([/web/20260426112138/https://ai.google.dev/gemini-api/docs/models/gemini-3.1-pro-preview#gemini-3.1-pro-preview-customtools](https://web.archive.org/web/20260426112138/https://ai.google.dev/gemini-api/docs/models/gemini-3.1-pro-preview#gemini-3.1-pro-preview-customtools)).

Next steps

- Get started with the [Gemini 3 Cookbook](https://web.archive.org/web/20260426112138/https://colab.research.google.com/github/google-gemini/cookbook/blob/main/quickstarts/Get_started.ipynb#templateParams=%7B%22MODEL_ID%22%3A+%22gemini-3-pro-preview%22%7D) (https://web.archive.org/web/20260426112138/https://colab.research.google.com/github/google-gemini/cookbook/blob/main/quickstarts/Get_started.ipynb#templateParams=%7B%22MODEL_ID%22%3A+%22gemini-3-pro-preview%22%7D)
- Check the dedicated Cookbook guide on [thinking levels](https://web.archive.org/web/20260426112138/https://colab.research.google.com/github/google-gemini/cookbook/blob/main/quickstarts/Get_started_thinking_REST.ipynb#gemini3) (https://web.archive.org/web/20260426112138/https://colab.research.google.com/github/google-gemini/cookbook/blob/main/quickstarts/Get_started_thinking_REST.ipynb#gemini3) and how to migrate from thinking budget to thinking levels.

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://web.archive.org/web/20260426112138/https://creativecommons.org/licenses/by/4.0/) (<https://web.archive.org/web/20260426112138/https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the [Apache 2.0 License](https://web.archive.org/web/20260426112138/https://www.apache.org/licenses/LICENSE-2.0) (<https://web.archive.org/web/20260426112138/https://www.apache.org/licenses/LICENSE-2.0>). For details, see the [Google Developers Site Policies](https://web.archive.org/web/20260426112138/https://developers.google.com/site-policies) (<https://web.archive.org/web/20260426112138/https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-04-14 UTC.

